VoetbalAI: wat heb ik getest en hoe werkt de app?

Een uitleg voor iedereen, ook als je nog nooit hebt geprogrammeerd. Getest op 27 oktober 2026.

1. Wat is VoetbalAI?

Een Android-app voor een voetbaltrainer. Je maakt een team, houdt bij wie er op training is en kunt chatten met een AI over de training. Dit is een **testversie**: het ziet er kaal uit, want de mooie schermen worden later gemaakt. Wat je nu ziet is een scherm om de **achterkant** (de “backend”) van de app uit te proberen.

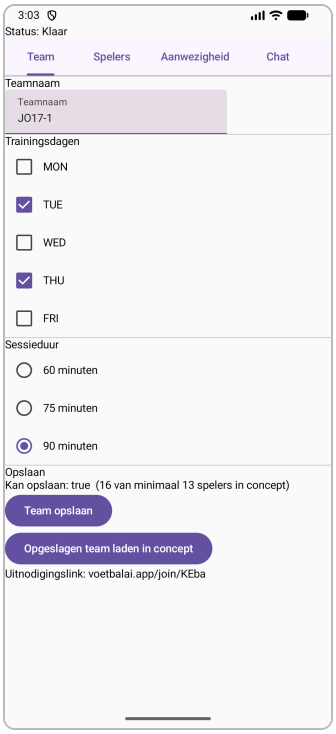
2. Hoe heb ik getest?

- **Op een nep-telefoon.** Ik heb de app gestart op de emulator in Android Studio (een telefoon op de computer, “Medium Phone”). Een echte telefoon heb ik niet gebruikt.
- **Schoon beginnen.** Ik wiste eerst de gegevens van de app, zodat het was alsof je de app voor het eerst opent.
- **Alle vijf tabbladen.** Team, Spelers, Aanwezigheid, Chat en Debug. Per tabblad heb ik de knoppen gebruikt en gekeken of dat gebeurde wat ik verwachtte.
- **Ook automatische tests.** Naast het klikken zijn er 47 kleine testprogramma’s (“unit tests”) die de regels van de app controleren. Ze slagen allemaal.

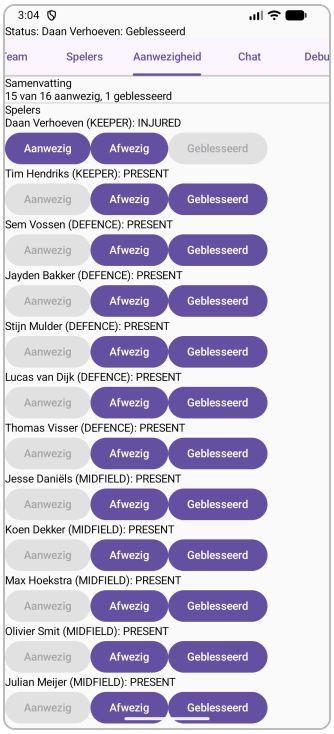
3. Wat gebeurde er?

Wat deed ik?	Wat gebeurde er?	Gelukt?
App voor het eerst openen	Teamnaam JO17-1, dinsdag en donderdag aangevinkt, 90 minuten. Er stond “16 van minimaal 13 spelers”. De testspelers worden dus vanzelf aangemaakt.	**Ja**
“Team opslaan” drukken	Bovenin stond “Team opgeslagen: JO17-1”.	**Ja**
Tabblad Spelers bekijken	16 spelers, per positie geteld: 2 keepers, 5 verdedigers, 5 middenvelders, 4 aanvallers.	**Ja**
Een speler op “Geblesseerd” zetten	De samenvatting veranderde meteen van “16 van 16 aanwezig” naar “15 van 16 aanwezig, 1 geblesseerd”.	**Ja**
Chat: “Wie speelt er zaterdag” typen en op Verstuur drukken	Eerst stond mijn bericht in het gesprek, daarna kwam het antwoord: “De chat-API is nog niet gekoppeld…”. Dat antwoord is **nep**, want de echte AI is er nog niet.	**Half (nep-antwoord)**

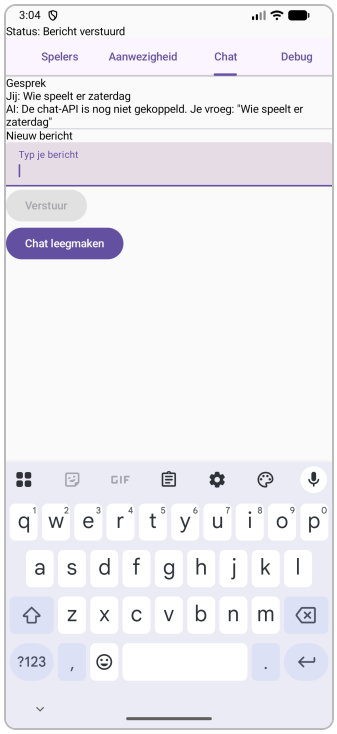
4 spelers weghalen (dan zijn er 12) en opslaan	De app weigerde en zei: “Minimaal 13 spelers nodig, er zijn er 12. Minimaal 1 keeper nodig, er zijn er 0.” De regels werken dus.	**Ja**
Debug: “Database resetten”	Bovenin stond “Database gereset” en bij Aanwezigheid stond “0 van 0”. Alle opgeslagen spelers waren weg.	**Ja**
Debug: “Seed opnieuw laden”	De 16 testspelers waren weer terug.	**Ja**
Speler “Test Speler” direct aan het opgeslagen team toevoegen	Het aantal ging naar 17.	**Ja**
App helemaal afsluiten en opnieuw openen	De aanwezigheid (Daan Verhoeven afwezig) en de extra speler stonden er nog. Alles wordt dus echt bewaard.	**Ja**



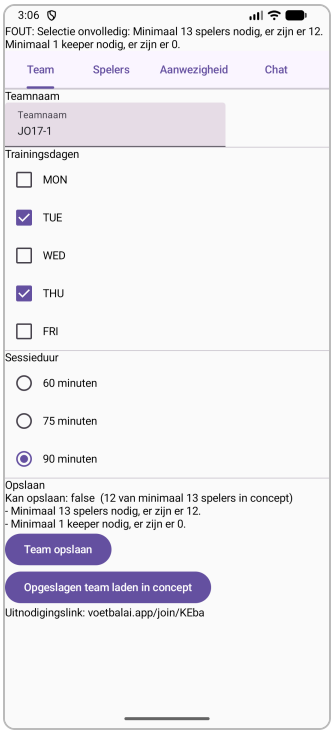
Team



Aanwezigheid



Chat



Foutmelding bij 12 spelers

4. Wat viel me op?

> **Waarom stond er eerder “0 van 0 aanwezig”?** Dat komt door de knop “Database resetten” in het tabblad Debug. Die wist alle opgeslagen spelers. Met “Seed opnieuw laden” kom je weer terug bij de 16.

> **Waarom stonden er toch 16 spelers bij “Spelers”?** Dat is het *concept*: het team dat je aan het invullen bent. Dat staat in het geheugen van de app en blijft staan als je de database wist. Daarom stond er ook “Kan opslaan: true” terwijl er niets was opgeslagen. Dat is zo bedoeld, maar kan verwarrend zijn.

- **De foutmelding blijft staan.** Na de melding “Selectie onvolledig” bleef de rode tekst bovenin staan, ook toen ik naar andere tabbladen ging. Hij verdwijnt pas als je in het Team-gedeelte iets anders doet. Een klein ongemak van het testscherm, geen crash.
- **De uitnodigingslink verandert** bij elke keer dat je de app start (bijvoorbeeld `voetbalai.app/join/KEba`). Dat is zo gemaakt: elke keer een nieuwe willekeurige code. De link werkt nog niet echt, want er is geen server.
- **Het tabblad Debug staat half buiten beeld.** Het tabbalkje scrolt opzij. Op een kleine telefoon zie je Debug pas als je naar rechts veegt.
- **Crashes:** in de foutenlijst van Android (logcat) stond geen crash van de app.

5. Wat heb ik niet getest?

- Een echte telefoon (alleen de emulator).
- Het scherm draaien (liggend/staand) en hele grote letters.
- De echte AI. Die is er nog niet.
- De mooie schermen. Die zijn er nog niet.

6. Hoe is de app gebouwd?

De app bestaat uit vier lagen. Elke laag doet één ding. Zo kun je één laag vervangen zonder dat de rest kapotgaat.

ui (het scherm): knoppen, tekst en tabbladen. Hier kijkt de gebruiker naar.

viewmodel (het geheugen van een scherm): onthoudt wat er op het scherm staat en wat er gebeurt als je op een knop drukt.

domain (het voetbalwereldje): wat is een speler of team, welke regels gelden? Hier zit geen Android in. Deze laag kent de database niet.

data (de opslag): de database op de telefoon. Deze laag *kent* het voetbalwereldje en doet wat daar beloofd wordt.

De pijlen: `ui → viewmodel → domain ← data`. Let op de laatste pijl: **data** wijst naar **domain**, niet andersom. Het voetbalwereldje zegt wat het nodig heeft (een “belofte”, een interface) en de opslag zorgt dat het waargemaakt wordt.

In de map `app/src/main/java/com/example/voetbalai/` zie je dat terug: `ui/`, `viewmodel/`, `domain/`, `data/`.

7. Waarom deze keuzes?

Keuze	Waarom?
Kotlin + Jetpack Compose	Kotlin is de taal die Google aanraadt voor Android. Met Compose schrijf je een scherm als code.
Room-database	Een database op de telefoon, ook zonder internet. Room maakt van een gewone database (SQLite) simpele Kotlin-code.
Alles lokaal (versie 1)	Geen server en geen online database. Dat is makkelijker om mee te beginnen. Een server komt later.
Interface (een belofte)	`TeamRepository` en `ChatApi` zeggen alleen *wat* iets kan. Zo kan ik een nepje gebruiken om te testen en later de echte AI erachter zetten.
Nep-chat	De echte AI is er nog niet. Met een nepantwoord kan ik de chat al bouwen en testen. Later verander ik één regel in `AppContainer.kt`.
Concept eerst, dan opslaan	Je vult eerst een concept in het geheugen. Pas bij “Team opslaan” gaat het naar de database. Zo staat er nooit een half team in de opslag.
Teamregels	Een voetbalteam heeft 11 spelers plus wissels, dus de app eist minimaal 13 spelers en van elke positie een minimum (1 keeper, 3 verdedigers, 3 middenvelders, 2 aanvallers).
Engelse namen in de code	Alle namen in de code zijn Engels. Wat je op het scherm leest en de uitleg in de code is Nederlands.

8. De database

De database heet `voetbalai.db` en staat op de telefoon. Denk aan twee Excel-bladen:

teams (een team)	players (een speler)
id: uniek nummer (de database verzint het zelf) name: bijvoorbeeld JO17-1 trainingDays: als tekst, bijvoorbeeld TUE,THU sessionMinutes: bijvoorbeeld 90 inviteCode: voor de uitnodigingslink	id: uniek nummer teamId: bij welk team hoort hij? name: bijvoorbeeld Daan Verhoeven position: KEEPER, DEFENCE, MIDFIELD of ATTACK attendance: PRESENT, ABSENT of INJURED

Een team heeft veel spelers (`teamId` wijst naar het `id` van het team).

- **Entity** = één rij in een tabel (`TeamEntity`, `PlayerEntity`).
- **DAO** = de lijst met opdrachten voor de database: opslaan, ophalen, aanpassen, verwijderen. Ik schrijf alleen de regels, Room maakt de echte code.
- **Flow** = een stroom die meekijkt. Verandert er iets in de database, dan komt er vanzelf een nieuwe waarde en ververst het scherm.
- **Transactie** = alles lukt, of niets. Team opslaan vervangt het oude team en zijn spelers in één keer, dus er blijft nooit een half team staan.
- **Converter**: de database bewaart alleen simpele dingen, dus een positie wordt opgeslagen als tekst (“KEEPER”) en bij het lezen teruggezet.

- **Mapper**: vertaalt tussen de database-rij en het object dat de app gebruikt.
- **Migratie**: de database ging van versie 1 naar 2 omdat de kolom `attendance` erbij kwam, zonder dat gegevens verdwijnen.
- **Seed** = testdata. Bij de eerste start komt een verzonnen team met 16 spelers in de database.
- **SharedPreferences**: een klein bestandje voor één ja/nee-ding: is het team opzetten al klaar?

9. Wat doet welk bestand?

Bestand	Wat doet het?
VoetbalApp.kt	Wordt één keer gemaakt als de app start. Maakt de AppContainer.
AppContainer.kt	Kiest welke echte onderdelen de app gebruikt: de Room-opslag en de nep-chat.
domain/model/Models.kt	De basisdingen: Player, Team, Position, Attendance, Day.
domain/rules/TeamRules.kt	De regels voor een geldig team. Geeft een lijstje met wat er nog ontbreekt.
domain/schedule/TrainingSchedule.kt	Rekent uit wanneer de volgende training is en welke begroeting past.
domain/chat/Chat.kt	Alles rond de chat: wat stuur ik, wat komt terug, en het nepantwoord.
domain/repository/TeamRepository.kt	De belofte: dit kan de opslag.
data/repository/RoomTeamRepository.kt	De echte uitvoering van die belofte, met de database.
data/local/…	De database zelf: tabellen, opdrachten en het opzetten.
viewmodel/…	Per scherm een “geheugen”: onboarding (team opzetten), today (vandaag), chat, app, welcome.
ui/test/…	Het kale testscherm met de vijf tabbladen die ik getest heb.

10. Voorbeeld: wat gebeurt er als je in de chat op Verstuur drukt?

1. Het scherm roept `onMessageSent(text)` aan bij de ChatViewModel.
2. Een leeg bericht, of een bericht terwijl er nog op een antwoord gewacht wordt, negeert de app.
3. Jouw bericht komt meteen in het gesprek te staan (“Jij: …”).
4. De app haalt je team op uit de database (naam, trainingsduur, spelers).
5. Het gesprek en de teamgegevens gaan naar de `ChatApi`. Nu is dat de nepversie, straks de echte AI. Zo weet de AI over welk team het gaat.
6. Het antwoord komt onder het gesprek. Gaat er iets mis, dan krijg je een foutmelding.

Voorbeeld: een team opslaan

Je werkt eerst in een concept (alleen in het geheugen). Bij “Team opslaan” controleert de app: is er een teamnaam, is er minstens één trainingsdag en voldoet de selectie aan de teamregels? Zo niet, dan krijg je een duidelijke melding, bijvoorbeeld *Minimaal 3 verdedigers nodig, er zijn er 2.*